

Informe de resultados DB SHOP

La actividad continua la base DB_SHOP de la Actividad 2. La solución fortalece el modelo, aplica seguridad por roles, incorpora reportes y automatiza la integración de pedidos del canal Tienda Online.

Alcance

El archivo Jorge Salazar.sql está organizado en los seis bloques solicitados. Las capturas siguientes muestran los objetos y consultas que se ejecutan sobre SQL Server. Antes de ejecutar BULK INSERT se debe adaptar la ruta C:\ETL\pedidos_online.csv y habilitar el acceso de la cuenta de SQL Server a esa carpeta.

Evidencia - 01 Portada Modelo

```

BLOQUE 1 - Fortalecimiento del modelo: esquemas e integridad
===== */
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'ventas') EXEC('CREATE SCHEMA ventas');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'catalogo') EXEC('CREATE SCHEMA catalogo');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'personas') EXEC('CREATE SCHEMA personas');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'staging') EXEC('CREATE SCHEMA staging');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'etl') EXEC('CREATE SCHEMA etl');
GO

-- Se trasladan las tablas de dbo sin recrear datos existentes.
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL ALTER SCHEMA catalogo TRANSFER dbo.Categories;
IF OBJECT_ID('dbo.FamilyCategories', 'U') IS NOT NULL ALTER SCHEMA catalogo TRANSFER dbo.FamilyCategories;
IF OBJECT_ID('dbo.Product', 'U') IS NOT NULL ALTER SCHEMA catalogo TRANSFER dbo.Product;
IF OBJECT_ID('dbo.Suppliers', 'U') IS NOT NULL ALTER SCHEMA catalogo TRANSFER dbo.Suppliers;
IF OBJECT_ID('dbo.Customers', 'U') IS NOT NULL ALTER SCHEMA personas TRANSFER dbo.Customers;
IF OBJECT_ID('dbo.Employees', 'U') IS NOT NULL ALTER SCHEMA personas TRANSFER dbo.Employees;
IF OBJECT_ID('dbo.Orders', 'U') IS NOT NULL ALTER SCHEMA ventas TRANSFER dbo.Orders;
IF OBJECT_ID('dbo.OrdersDetails', 'U') IS NOT NULL ALTER SCHEMA ventas TRANSFER dbo.OrdersDetails;
IF OBJECT_ID('dbo.Shippers', 'U') IS NOT NULL ALTER SCHEMA ventas TRANSFER dbo.Shippers;
IF OBJECT_ID('dbo.TypeOrders', 'U') IS NOT NULL ALTER SCHEMA ventas TRANSFER dbo.TypeOrders;
IF OBJECT_ID('dbo.MarketingCampaigns', 'U') IS NOT NULL ALTER SCHEMA ventas TRANSFER dbo.MarketingCampaigns;
GO

-- La importacion original no tenia llave en el detalle del pedido.
IF NOT EXISTS (SELECT 1 FROM sys.key_constraints WHERE parent_object_id = OBJECT_ID('ventas.OrdersDetails') AND type = 'PK')
    ALTER TABLE ventas.OrdersDetails ADD CONSTRAINT PK_OrdersDetails PRIMARY KEY (OrderID, ProductID);
GO

-- Llaves foraneas. Los nombres impiden crear duplicados si se vuelve a ejecutar el script.
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Categories_FamilyCategories')
    ALTER TABLE catalogo.Categories ADD CONSTRAINT FK_Categories_FamilyCategories FOREIGN KEY (FamilyID) REFERENCES catalogo.FamilyCategories(FamilyID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Product_Categories')
    ALTER TABLE catalogo.Product ADD CONSTRAINT FK_Product_Categories FOREIGN KEY (CategoryID) REFERENCES catalogo.Categories(CategoryID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Product_Suppliers')
    ALTER TABLE catalogo.Product ADD CONSTRAINT FK_Product_Suppliers FOREIGN KEY (SupplierID) REFERENCES catalogo.Suppliers(SupplierID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Employees_Employees')
    ALTER TABLE personas.Employees ADD CONSTRAINT FK_Employees_Employees FOREIGN KEY (ReportsTo) REFERENCES personas.Employees(EmployeeID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_Customers')
    ALTER TABLE ventas.Orders ADD CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerID) REFERENCES personas.Customers(CustomerID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_Employees')
    ALTER TABLE ventas.Orders ADD CONSTRAINT FK_Orders_Employees FOREIGN KEY (EmployeeID) REFERENCES personas.Employees(EmployeeID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_TypeOrders')
    ALTER TABLE ventas.Orders ADD CONSTRAINT FK_Orders_TypeOrders FOREIGN KEY (TypeOrderID) REFERENCES ventas.TypeOrders(TypeOrderID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_Shippers')
    ALTER TABLE ventas.Orders ADD CONSTRAINT FK_Orders_Shippers FOREIGN KEY (ShipVia) REFERENCES ventas.Shippers(ShipperID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_Orders_Campaigns')
    ALTER TABLE ventas.Orders ADD CONSTRAINT FK_Orders_Campaigns FOREIGN KEY (CampaignID) REFERENCES ventas.MarketingCampaigns(CampaignID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_OrdersDetails_Orders')
    ALTER TABLE ventas.OrdersDetails ADD CONSTRAINT FK_OrdersDetails_Orders FOREIGN KEY (OrderID) REFERENCES ventas.Orders(OrderID);
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_OrdersDetails_Product')
    ALTER TABLE ventas.OrdersDetails ADD CONSTRAINT FK_OrdersDetails_Product FOREIGN KEY (ProductID) REFERENCES catalogo.Product(ProductID);
GO

/* =====

```

Captura 1. Esquemas, traslado de tablas e integridad referencial.

Seguridad basada en roles

Se crean cuatro logins, usuarios y roles. Los permisos se entregan por esquema aplicando mínimo privilegio y se incluyen pruebas con EXECUTE AS USER.

Evidencia - 02 Seguridad

```

BLOQUE 2 - Seguridad basada en roles
===== */
USE master;
GO
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'ventas1') CREATE LOGIN ventas1 WITH PASSWORD = 'Ventas#2026Segura', CHECK_POLICY = ON
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'catalogo1') CREATE LOGIN catalogo1 WITH PASSWORD = 'Catalogo#2026Segura', CHECK_POLIC
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'analista1') CREATE LOGIN analista1 WITH PASSWORD = 'Analista#2026Segura', CHECK_POLIC
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'etl_service') CREATE LOGIN etl_service WITH PASSWORD = 'Etl#2026Servicio', CHECK_POLI
GO
USE DB_SHOP;
GO
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'ventas1') CREATE USER ventas1 FOR LOGIN ventas1;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'catalogo1') CREATE USER catalogo1 FOR LOGIN catalogo1;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'analista1') CREATE USER analista1 FOR LOGIN analista1;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'etl_service') CREATE USER etl_service FOR LOGIN etl_service;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_ventas') CREATE ROLE rol_ventas;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_catalogo') CREATE ROLE rol_catalogo;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_analista') CREATE ROLE rol_analista;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_etl') CREATE ROLE rol_etl;
ALTER ROLE rol_ventas ADD MEMBER ventas1;
ALTER ROLE rol_catalogo ADD MEMBER catalogo1;
ALTER ROLE rol_analista ADD MEMBER analista1;
ALTER ROLE rol_etl ADD MEMBER etl_service;
GO
GRANT SELECT, INSERT, UPDATE ON SCHEMA::ventas TO rol_ventas;
DENY DELETE ON SCHEMA::ventas TO rol_ventas;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::catalogo TO rol_catalogo;
DENY DELETE ON SCHEMA::catalogo TO rol_catalogo;
GRANT SELECT ON SCHEMA::ventas TO rol_analista;
GRANT SELECT ON SCHEMA::catalogo TO rol_analista;
GRANT SELECT ON SCHEMA::personas TO rol_analista;
DENY INSERT, UPDATE, DELETE ON SCHEMA::ventas TO rol_analista;
DENY INSERT, UPDATE, DELETE ON SCHEMA::catalogo TO rol_analista;
DENY INSERT, UPDATE, DELETE ON SCHEMA::personas TO rol_analista;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::staging TO rol_etl;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::etl TO rol_etl;
GRANT EXECUTE ON SCHEMA::etl TO rol_etl;
GO
-- Pruebas de permisos: se valida que el analista pueda leer y que ventas pueda insertar.
EXECUTE AS USER = 'analista1'; SELECT TOP (1) * FROM ventas.Orders; REVERT;
EXECUTE AS USER = 'ventas1'; SELECT HAS_PERMS_BY_NAME('ventas.Orders', 'OBJECT', 'INSERT') AS PuedeInsertar, HAS_PERMS_BY_NAME('ventas.Orders', 'OBJEC
GO

/* =====

```

Captura 2. Evidencia del bloque implementado.

Vistas de reporte

Las vistas resumen ventas por cliente, productos más vendidos, pedidos pendientes e inventario para reabastecer.

Evidencia - 03 Vistas

```
BLOQUE 3 - Vistas de reporte
===== */
CREATE OR ALTER VIEW ventas.vw_ResumenVentasPorCliente AS
SELECT c.CustomerID, c.CompanyName, COUNT(DISTINCT o.OrderID) AS NumeroPedidos,
      CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(18,2)) AS TotalComprado
FROM personas.Customers c
JOIN ventas.Orders o ON o.CustomerID = c.CustomerID
JOIN ventas.OrdersDetails od ON od.OrderID = o.OrderID
GROUP BY c.CustomerID, c.CompanyName;
GO

CREATE OR ALTER VIEW catalogo.vw_ProductosMasVendidos AS
SELECT p.ProductID, p.ProductName, SUM(od.Quantity) AS UnidadesVendidas,
      CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(18,2)) AS IngresoGenerado
FROM catalogo.Product p JOIN ventas.OrdersDetails od ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName;
GO

CREATE OR ALTER VIEW ventas.vw_PedidosPendientesEnvio AS
SELECT o.OrderID, o.OrderDate, c.CompanyName AS Cliente, s.CompanyName AS Transportista
FROM ventas.Orders o JOIN personas.Customers c ON c.CustomerID = o.CustomerID
LEFT JOIN ventas.Shippers s ON s.ShipperID = o.ShipVia
WHERE o.ShippedDate IS NULL;
GO

CREATE OR ALTER VIEW catalogo.vw_InventarioParaReabastecer AS
SELECT ProductID, ProductName, UnitsInStock, ReorderLevel, UnitsOnOrder
FROM catalogo.Product WHERE UnitsInStock <= ReorderLevel;
GO

/* =====
```

Captura 3. Evidencia del bloque implementado.

Procedimientos de mantenimiento

Se implementan INSERT, UPDATE y DELETE con validaciones. Los ejemplos comentados permiten probar cada procedimiento con identificadores existentes.

Evidencia - 04 Procedimientos

```
BLOQUE 4 - Procedimientos almacenados de mantenimiento
===== */
CREATE OR ALTER PROCEDURE ventas.sp_InsertarPedido
    @CustomerID NVARCHAR(255), @EmployeeID INT, @OrderDate DATE, @RequiredDate DATE = NULL,
    @ShipVia INT = NULL, @Freight MONEY = 0, @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM personas.Customers WHERE CustomerID = @CustomerID) THROW 50001, 'Cliente inexistente.', 1;
    IF NOT EXISTS (SELECT 1 FROM personas.Employees WHERE EmployeeID = @EmployeeID) THROW 50002, 'Empleado inexistente.', 1;
    INSERT INTO ventas.Orders (CustomerID, EmployeeID, OrderDate, RequiredDate, ShipVia, Freight)
    VALUES (@CustomerID, @EmployeeID, @OrderDate, @RequiredDate, @ShipVia, @Freight);
    SET @OrderID = CONVERT(INT, SCOPE_IDENTITY());
END;
GO
CREATE OR ALTER PROCEDURE ventas.sp_InsertarDetallePedido
    @OrderID INT, @ProductID INT, @Quantity SMALLINT, @UnitPrice MONEY = NULL, @Discount REAL = 0
AS
BEGIN
    SET NOCOUNT ON;
    IF @Quantity IS NULL OR @Quantity <= 0 THROW 50003, 'La cantidad debe ser mayor que cero.', 1;
    IF @Discount < 0 OR @Discount >= 1 THROW 50004, 'El descuento debe estar entre 0 y menor que 1.', 1;
    IF NOT EXISTS (SELECT 1 FROM ventas.Orders WHERE OrderID = @OrderID) THROW 50005, 'Pedido inexistente.', 1;
    IF NOT EXISTS (SELECT 1 FROM catalogo.Product WHERE ProductID = @ProductID) THROW 50006, 'Producto inexistente.', 1;
    SELECT @UnitPrice = COALESCE(@UnitPrice, UnitPrice) FROM catalogo.Product WHERE ProductID = @ProductID;
    INSERT INTO ventas.OrdersDetails (OrderID, ProductID, UnitPrice, Quantity, Discount)
    VALUES (@OrderID, @ProductID, @UnitPrice, @Quantity, @Discount);
END;
```

Captura 4. Evidencia del bloque implementado.

Staging y control ETL

La tabla staging conserva el texto original del CSV. Las tablas etl.pedidos_rechazados, etl.pedidos_cargados y etl.log_ejecucion permiten auditar cada carga.

Evidencia - 05 Staging

```
CREATE OR ALTER PROCEDURE ventas.sp_ActualizarEstadoEnvio
    @OrderID INT, @ShipperID INT, @FechaEnvio DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM ventas.Shippers WHERE ShipperID = @ShipperID) THROW 50007, 'Transportista inexistente.', 1;
    IF EXISTS (SELECT 1 FROM ventas.Orders WHERE OrderID = @OrderID AND ShippedDate IS NOT NULL) THROW 50008, 'El pedido ya fue enviado.', 1;
    UPDATE ventas.Orders SET ShippedDate = COALESCE(@FechaEnvio, CONVERT(DATE, GETDATE())), ShipVia = @ShipperID WHERE OrderID = @OrderID;
    IF @@ROWCOUNT = 0 THROW 50009, 'Pedido inexistente.', 1;
END;
GO

CREATE OR ALTER PROCEDURE catalogo.sp_ActualizarPrecioProducto
    @CategoryID INT, @Porcentaje DECIMAL(9,4)
AS
BEGIN
    SET NOCOUNT ON;
    IF @Porcentaje <= -100 THROW 50010, 'El porcentaje no puede reducir el precio a cero o menos.', 1;
    UPDATE catalogo.Product SET UnitPrice = ROUND(UnitPrice * (1 + @Porcentaje / 100.0), 2) WHERE CategoryID = @CategoryID;
    SELECT @@ROWCOUNT AS ProductosActualizados;
END;
GO

CREATE OR ALTER PROCEDURE ventas.sp_EliminarPedido @OrderID INT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM ventas.Orders WHERE OrderID = @OrderID AND ShippedDate IS NOT NULL) THROW 50011, 'No se puede eliminar un pedido enviado.', 1;
    IF NOT EXISTS (SELECT 1 FROM ventas.Orders WHERE OrderID = @OrderID) THROW 50012, 'Pedido inexistente.', 1;
    BEGIN TRANSACTION;
    DELETE FROM ventas.OrdersDetails WHERE OrderID = @OrderID;
    DELETE FROM ventas.Orders WHERE OrderID = @OrderID;
    COMMIT TRANSACTION;
END;
GO

-- Pruebas controladas: sustituir los identificadores por valores reales de la base.
-- DECLARE @NuevoPedido INT; EXEC ventas.sp_InsertarPedido 'ALFKI', 1, '2026-09-01', NULL, 1, 0, @NuevoPedido OUTPUT;
-- EXEC ventas.sp_InsertarDetallePedido @NuevoPedido, 1, 2, NULL, 0;
-- EXEC ventas.sp_ActualizarEstadoEnvio @NuevoPedido, 1, '2026-09-02';
-- EXEC catalogo.sp_ActualizarPrecioProducto 1, 5;
-- EXEC ventas.sp_EliminarPedido @NuevoPedido; -- Ejecutar solamente si no fue enviado.
GO

/* =====
```

Captura 5. Evidencia del bloque implementado.

Normalización y validación

El procedimiento identifica fechas con formatos mixtos, valores numéricos incorrectos, descuentos inválidos y clientes o productos no reconocidos.

Evidencia - 06 Etl Validation

```

BLOQUE 5 - Integracion ETL Tienda Online
===== */
-- Metadatos de trazabilidad para hacer re-ejecutable la carga.
IF COL_LENGTH('ventas.Orders', 'NroPedidoWeb') IS NULL
ALTER TABLE ventas.Orders ADD NroPedidoWeb NVARCHAR(50) NULL;
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'UQ_Orders_NroPedidoWeb' AND object_id = OBJECT_ID('ventas.Orders'))
CREATE UNIQUE INDEX UQ_Orders_NroPedidoWeb ON ventas.Orders(NroPedidoWeb) WHERE NroPedidoWeb IS NOT NULL;
GO
IF NOT EXISTS (SELECT 1 FROM ventas.TypeOrders WHERE TypeOrderName = 'Online Order')
INSERT INTO ventas.TypeOrders (TypeOrderName) VALUES ('Online Order');
IF NOT EXISTS (SELECT 1 FROM personas.Employees WHERE FirstName = 'Canal' AND LastName = 'Online')
INSERT INTO personas.Employees (FirstName, LastName, Title) VALUES ('Canal', 'Online', 'Canal de ventas');
GO
IF OBJECT_ID('staging.stg_pedidos_online', 'U') IS NULL
CREATE TABLE staging.stg_pedidos_online (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, NroPedidoWeb NVARCHAR(50) NULL, FechaPedido NVARCHAR(30) NULL,
    ClienteEmpresa NVARCHAR(255) NULL, Producto NVARCHAR(255) NULL, Cantidad NVARCHAR(30) NULL,
    PrecioUnitario NVARCHAR(30) NULL, Descuento NVARCHAR(30) NULL, MetodoEnvio NVARCHAR(150) NULL,
    Campana NVARCHAR(150) NULL, FechaCarga DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
IF OBJECT_ID('etl.pedidos_rechazados', 'U') IS NULL
CREATE TABLE etl.pedidos_rechazados (
    RechazoID BIGINT IDENTITY PRIMARY KEY, StagingID INT NOT NULL, NroPedidoWeb NVARCHAR(50), MotivoRechazo NVARCHAR(500) NOT NULL,
    FechaRechazo DATETIME2 NOT NULL DEFAULT SYSDATETIME(), CONSTRAINT UQ_Rechazo_Staging UNIQUE(StagingID)
);
IF OBJECT_ID('etl.pedidos_cargados', 'U') IS NULL
CREATE TABLE etl.pedidos_cargados (
    CargaID BIGINT IDENTITY PRIMARY KEY, StagingID INT NOT NULL, NroPedidoWeb NVARCHAR(50) NOT NULL, OrderID INT NOT NULL,
    FechaCarga DATETIME2 NOT NULL DEFAULT SYSDATETIME(), CONSTRAINT UQ_Cargado_Staging UNIQUE(StagingID)
);
IF OBJECT_ID('etl.log_ejecucion', 'U') IS NULL
CREATE TABLE etl.log_ejecucion (
    EjecucionID BIGINT IDENTITY PRIMARY KEY, Inicio DATETIME2 NOT NULL, Fin DATETIME2 NULL, TotalStaging INT NULL,
    Cargados INT NULL, Rechazados INT NULL, Estado NVARCHAR(20) NOT NULL, Detalle NVARCHAR(1000) NULL
);
GO
-- Ajustar la ruta para el servidor SQL. La cuenta del servicio debe poder leer esa carpeta.
-- TRUNCATE TABLE staging.stg_pedidos_online;
-- BULK INSERT staging.stg_pedidos_online FROM 'C:\\ETL\\pedidos_online.csv'
-- WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
GO

```

Captura 6. Evidencia del bloque implementado.

Carga transaccional

Las filas validas se agrupan por pedido web y se insertan por conjuntos en Orders y OrdersDetails. La marca NroPedidoWeb y las tablas de control hacen la ejecución idempotente.

Evidencia - 07 Etl Carga

```
CREATE OR ALTER PROCEDURE etl.sp_CargarPedidosOnline
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @EjecucionID BIGINT, @Inicio DATETIME2 = SYSDATETIME();
    INSERT INTO etl.log_ejecucion (Inicio, Estado) VALUES (@Inicio, 'EN PROCESO');
    SET @EjecucionID = SCOPE_IDENTITY();

    BEGIN TRY
        BEGIN TRANSACTION;
        ;WITH Normalizado AS (
            SELECT s.*, COALESCE(TRY_CONVERT(date, FechaPedido, 23), TRY_CONVERT(date, FechaPedido, 103), TRY_CONVERT(date, FechaPedido, 101), TRY_CONVERT(date,
                TRY_CONVERT(INT, Cantidad) AS CantidadOk,
                TRY_CONVERT(DECIMAL(12,2), REPLACE(PrecioUnitario, ',', '.')) AS PrecioOk,
                COALESCE(TRY_CONVERT(DECIMAL(5,4), REPLACE(Descuento, ',', '.')), 0) AS DescuentoOk,
                c.CustomerID, p.ProductID, sh.ShipperID, mc.CampaingID
            FROM staging.stg_pedidos_online s
            LEFT JOIN personas.Customers c ON UPPER(LTRIM(RTRIM(c.CompanyName))) = UPPER(LTRIM(RTRIM(s.ClienteEmpresa)))
            LEFT JOIN catalogo.Product p ON UPPER(LTRIM(RTRIM(p.ProductName))) = UPPER(LTRIM(RTRIM(s.Producto)))
            LEFT JOIN ventas.Shippers sh ON UPPER(LTRIM(RTRIM(sh.CompanyName))) = UPPER(LTRIM(RTRIM(s.MetodoEnvio)))
            LEFT JOIN ventas.MarketingCampaings mc ON UPPER(LTRIM(RTRIM(mc.CampaingName))) = UPPER(LTRIM(RTRIM(s.Campana)))
        )
        INSERT INTO etl.pedidos_rechazados (StagingID, NroPedidoWeb, MotivoRechazo)
        SELECT StagingID, NroPedidoWeb,
            CONCAT(CASE WHEN NULLIF(LTRIM(RTRIM(NroPedidoWeb)), '') IS NULL THEN 'NroPedidoWeb vacio; ' ELSE '' END,
                CASE WHEN FechaOk IS NULL THEN 'Fecha invalida; ' ELSE '' END,
                CASE WHEN CantidadOk IS NULL OR CantidadOk <= 0 THEN 'Cantidad invalida; ' ELSE '' END,
                CASE WHEN PrecioOk IS NULL OR PrecioOk < 0 THEN 'Precio invalido; ' ELSE '' END,
                CASE WHEN DescuentoOk < 0 OR DescuentoOk >= 1 THEN 'Descuento invalido; ' ELSE '' END,
                CASE WHEN CustomerID IS NULL THEN 'Cliente no reconocido; ' ELSE '' END,
                CASE WHEN ProductID IS NULL THEN 'Producto no reconocido; ' ELSE '' END)
        FROM Normalizado n
        WHERE NOT EXISTS (SELECT 1 FROM etl.pedidos_cargados pc WHERE pc.StagingID = n.StagingID)
        AND NOT EXISTS (SELECT 1 FROM etl.pedidos_rechazados pr WHERE pr.StagingID = n.StagingID)
        AND (NULLIF(LTRIM(RTRIM(NroPedidoWeb)), '') IS NULL OR FechaOk IS NULL OR CantidadOk IS NULL OR CantidadOk <= 0 OR PrecioOk IS NULL OR PrecioOk <
            0 OR DescuentoOk < 0 OR DescuentoOk >= 1)

        SELECT n.StagingID, n.NroPedidoWeb, n.FechaOk, n.CustomerID, n.ProductID, n.CantidadOk, n.PrecioOk, n.DescuentoOk, n.ShipperID, n.CampaingID
        INTO #Validos
        FROM (
            SELECT s.*, COALESCE(TRY_CONVERT(date, FechaPedido, 23), TRY_CONVERT(date, FechaPedido, 103), TRY_CONVERT(date, FechaPedido, 101), TRY_CONVERT(date,
```

Captura 7. Evidencia del bloque implementado.

Reconciliación y rechazos

Las consultas comparan las filas de staging contra cargadas y rechazadas. El conteo debe cerrar con diferencia cero y el agrupamiento identifica los motivos frecuentes.

Evidencia - 08 Reconciliation

```

    TRY_CONVERT(INT, Cantidad) AS CantidadOk, TRY_CONVERT(DECIMAL(12,2), REPLACE(PrecioUnitario, ',', '.')) AS PrecioOk,
    COALESCE(TRY_CONVERT(DECIMAL(5,4), REPLACE(Descuento, ',', '.')), 0) AS DescuentoOk, c.CustomerID, p.ProductID, sh.ShipperID, mc.CampaingID
FROM staging.stg_pedidos_online s
LEFT JOIN personas.Customers c ON UPPER(LTRIM(RTRIM(c.CompanyName))) = UPPER(LTRIM(RTRIM(s.ClienteEmpresa)))
LEFT JOIN catalogo.Product p ON UPPER(LTRIM(RTRIM(p.ProductName))) = UPPER(LTRIM(RTRIM(s.Producto)))
LEFT JOIN ventas.Shippers sh ON UPPER(LTRIM(RTRIM(sh.CompanyName))) = UPPER(LTRIM(RTRIM(s.MetodoEnvio)))
LEFT JOIN ventas.MarketingCampaigns mc ON UPPER(LTRIM(RTRIM(mc.CampaingName))) = UPPER(LTRIM(RTRIM(s.Campana)))
) n
WHERE n.FechaOk IS NOT NULL AND n.CantidadOk > 0 AND n.PrecioOk >= 0 AND n.DescuentoOk >= 0 AND n.DescuentoOk < 1
AND n.CustomerID IS NOT NULL AND n.ProductID IS NOT NULL
AND NOT EXISTS (SELECT 1 FROM etl.pedidos_cargados pc WHERE pc.StagingID = n.StagingID)
AND NOT EXISTS (SELECT 1 FROM etl.pedidos_rechazados pr WHERE pr.StagingID = n.StagingID);

DECLARE @TipoOnline INT = (SELECT TypeOrderID FROM ventas.TypeOrders WHERE TypeOrderName = 'Online Order');
DECLARE @EmpleadoOnline INT = (SELECT EmployeeID FROM personas.Employees WHERE FirstName = 'Canal' AND LastName = 'Online');
INSERT INTO ventas.Orders (CustomerID, EmployeeID, OrderDate, ShipVia, TypeOrderID, CampaingID, NroPedidoWeb)
SELECT v.CustomerID, @EmpleadoOnline, MIN(v.FechaOk), MAX(v.ShipperID), @TipoOnline, MAX(v.CampaingID), v.NroPedidoWeb
FROM #Validos v
WHERE NOT EXISTS (SELECT 1 FROM ventas.Orders o WHERE o.NroPedidoWeb = v.NroPedidoWeb)
GROUP BY v.CustomerID, v.NroPedidoWeb;

INSERT INTO ventas.OrdersDetails (OrderID, ProductID, UnitPrice, Quantity, Discount)
SELECT o.OrderID, v.ProductID, v.PrecioOk, v.CantidadOk, v.DescuentoOk
FROM #Validos v JOIN ventas.Orders o ON o.NroPedidoWeb = v.NroPedidoWeb
WHERE NOT EXISTS (SELECT 1 FROM ventas.OrdersDetails od WHERE od.OrderID = o.OrderID AND od.ProductID = v.ProductID);

INSERT INTO etl.pedidos_cargados (StagingID, NroPedidoWeb, OrderID)
SELECT v.StagingID, v.NroPedidoWeb, o.OrderID FROM #Validos v JOIN ventas.Orders o ON o.NroPedidoWeb = v.NroPedidoWeb;
COMMIT TRANSACTION;
UPDATE etl.log_ejecucion SET Fin = SYSDATETIME(), TotalStaging = (SELECT COUNT(*) FROM staging.stg_pedidos_online),
    Cargados = (SELECT COUNT(*) FROM etl.pedidos_cargados), Rechazados = (SELECT COUNT(*) FROM etl.pedidos_rechazados), Estado = 'COMPLETADO'
WHERE EjecucionID = @EjecucionID;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    UPDATE etl.log_ejecucion SET Fin = SYSDATETIME(), Estado = 'ERROR', Detalle = ERROR_MESSAGE() WHERE EjecucionID = @EjecucionID;
    THROW;
END CATCH
END;
GO

```

Captura 8. Evidencia del bloque implementado.

Preguntas de negocio

El bloque final responde el total generado, los cinco mejores clientes, los clientes multicanal y los pedidos online pendientes.

Evidencia - 09 Negocio

```
-- Perfil y reconciliacion; el segundo resultado debe tener Diferencia = 0.
SELECT COUNT(*) AS FilasStaging, COUNT(DISTINCT CONCAT(NroPedidoWeb, '|', Producto)) AS FilasUnicas FROM staging.stg_pedidos_online;
SELECT (SELECT COUNT(*) FROM staging.stg_pedidos_online) AS Staging, (SELECT COUNT(*) FROM etl.pedidos_cargados) AS Cargados,
       (SELECT COUNT(*) FROM etl.pedidos_rechazados) AS Rechazados,
       (SELECT COUNT(*) FROM staging.stg_pedidos_online) - (SELECT COUNT(*) FROM etl.pedidos_cargados) - (SELECT COUNT(*) FROM etl.pedidos_rechazados) AS Di
SELECT MotivoRechazo, COUNT(*) AS Cantidad FROM etl.pedidos_rechazados GROUP BY MotivoRechazo ORDER BY Cantidad DESC;
-- EXEC etl.sp_CargarPedidosOnline;
GO

/* =====
```

Captura 9. Evidencia del bloque implementado.

Conclusiones

La solución protege la integridad referencial, separa el acceso por responsabilidades y deja una carga repetible y auditable para Tienda Online. Al ejecutar el script, los resultados de las consultas de reconciliación y negocio se actualizan con los datos reales de DB_SHOP.

Evidencia - 10 Resultados

```
BLOQUE 6 - Preguntas de negocio del canal online
===== */
-- 1. Ventas generadas por el canal online.
SELECT CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(18,2)) AS VentasCanalOnline
FROM ventas.Orders o JOIN ventas.OrdersDetails od ON od.OrderID = o.OrderID
JOIN ventas.TypeOrders t ON t.TypeOrderID = o.TypeOrderID WHERE t.TypeOrderName = 'Online Order';

-- 2. Cinco clientes con mayor compra online.
SELECT TOP (5) c.CompanyName, CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(18,2)) AS TotalOnline
FROM ventas.Orders o JOIN ventas.OrdersDetails od ON od.OrderID = o.OrderID JOIN personas.Customers c ON c.CustomerID = o.CustomerID
JOIN ventas.TypeOrders t ON t.TypeOrderID = o.TypeOrderID WHERE t.TypeOrderName = 'Online Order'
GROUP BY c.CompanyName ORDER BY TotalOnline DESC;

-- 3. Clientes que compraron por otro canal y tambien por Online.
SELECT c.CustomerID, c.CompanyName FROM personas.Customers c
WHERE EXISTS (SELECT 1 FROM ventas.Orders o JOIN ventas.TypeOrders t ON t.TypeOrderID = o.TypeOrderID WHERE o.CustomerID = c.CustomerID AND t.TypeOrderName = 'Online Order')
AND EXISTS (SELECT 1 FROM ventas.Orders o JOIN ventas.TypeOrders t ON t.TypeOrderID = o.TypeOrderID WHERE o.CustomerID = c.CustomerID AND (t.TypeOrderName != 'Online Order'));

-- 4. Pedidos online aun pendientes de envio.
SELECT o.OrderID, o.NroPedidoWeb, o.OrderDate, c.CompanyName FROM ventas.Orders o
JOIN personas.Customers c ON c.CustomerID = o.CustomerID JOIN ventas.TypeOrders t ON t.TypeOrderID = o.TypeOrderID
WHERE t.TypeOrderName = 'Online Order' AND o.ShippedDate IS NULL ORDER BY o.OrderDate;
GO
```

Captura 10. Cierre del script y consultas de resultado.

Evidencia de ejecución del panel web

El panel se levantó con Docker Compose en http://localhost:8091/panel_sql/. Permite consultar cada bloque, copiarlo para SQL Server Management Studio y descargar el script completo sin guardar credenciales.

ACTIVIDAD INTEGRADORA 3

DB SHOP · Panel de ejecucion

SQL Server · 6 bloques · ETL Tienda Online

Ejecuta el deber por bloques

Selecciona un bloque, revisa el T-SQL y cópialo a SQL Server Management Studio. Se requiere la base de_shop de la Actividad 2 antes de iniciar.

Descargar script completo

1 Fortalecimiento del modelo

Crea esquemas, traslada las tablas y agrega PK/FK para proteger la integridad.

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'ventas') EXEC('CREATE SCHEMA ventas');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'catalogo') EXEC('CREATE SCHEMA catalogo');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'personas') EXEC('CREATE SCHEMA personas');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'staging') EXEC('CREATE SCHEMA staging');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'etl') EXEC('CREATE SCHEMA etl');
GO

-- Se trasladan las tablas de dbo sin recrear datos existentes.
IF OBJECT_ID('dbo.Categorias', 'U') IS NOT NULL
ALTER SCHEMA catalogo TRANSFER dbo.Categorias;
IF OBJECT_ID('dbo.Productos', 'U') IS NOT NULL
ALTER SCHEMA catalogo TRANSFER dbo.Productos;
GO
```

Copiar bloque

Marcar revisado

Pendiente de ejecutar en SQL Server.

2 Seguridad basada en roles

Crea logins, usuarios y roles con permisos mínimos y pruebas de acceso.

```
USE master;
GO
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'ventas1') CREATE LOGIN ventas1 WITH PASSWORD = 'Ventas#2026Segura', CHECK_POLICY = ON;
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'catalogo1') CREATE LOGIN catalogo1 WITH PASSWORD = 'Catalogo#2026Segura', CHECK_POLICY = ON;
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'analista1') CREATE LOGIN analista1 WITH PASSWORD = 'Analista#2026Segura', CHECK_POLICY = ON;
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'etl_service') CREATE LOGIN etl_service WITH PASSWORD = 'Etl#2026Servicio', CHECK_POLICY = ON;
GO
```

Copiar bloque

Marcar revisado

Pendiente de ejecutar en SQL Server.

3 Vistas de reporte

Crea cuatro vistas para ventas, productos, envíos e inventario.

```
CREATE OR ALTER VIEW
ventas.vw_ResumenVentasPorCliente AS
SELECT c.CustomerID, c.CompanyName, COUNT(DISTINCT o.OrderID) AS NumeroPedidos,
       CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(18,2)) AS TotalComprado
FROM personas.Customers c
JOIN ventas.Orders o ON o.CustomerID = c.CustomerID
JOIN ventas.OrdersDetails od ON od.OrderID = o.OrderID
GROUP BY c.CustomerID, c.CompanyName;
GO
CREATE OR ALTER VIEW
catalogo.vw_ProductosMasVendidos AS
SELECT p.ProductID, p.ProductName,
       SUM(od.Quantity) AS TotalVendido
```

Copiar bloque

Marcar revisado

Pendiente de ejecutar en SQL Server.

4 Procedimientos almacenados

5 Integración ETL

6 Preguntas de negocio

Captura 11. Panel web local de DB SHOP mostrando los seis bloques del deber.

Estructura y uso de Docker

La solución se concentra en un script T-SQL ordenado por los seis bloques solicitados. El panel web es una ayuda de navegación: permite revisar, copiar o descargar cada bloque sin mezclar la interfaz con la ejecución real de SQL Server.

- Jorge Salazar.sql: solución completa, organizada de Bloque 1 a Bloque 6.
- pedidos_online.csv: fuente externa que alimenta la zona staging del ETL.
- panel_sql: interfaz HTML, CSS y JavaScript que separa y presenta cada bloque del script.
- docker-compose.yml: publica el panel con Nginx en un contenedor local.
- evidencias: conserva las capturas usadas en este informe.

Por qué se usa Docker

Docker permite mostrar el panel de apoyo con Nginx sin instalar ni configurar un servidor web en el equipo. El archivo docker-compose.yml fija el puerto y el contenido a publicar, por lo que cualquier equipo con Docker puede abrir el mismo panel en http://localhost:8091/panel_sql/. Esto es más confiable que abrir archivos HTML de forma aislada, porque conserva las rutas relativas al script SQL y al CSV. El panel no sustituye SQL Server: el T-SQL debe ejecutarse en SSMS contra la DB_SHOP de la Actividad 2.

Archivos principales

Archivo	Responsabilidad en la solución
Jorge Salazar.sql	Resuelve el ejercicio: esquemas, integridad, roles, vistas, procedimientos, ETL y consultas de negocio.
panel_sql/index.html	Estructura visual del panel y las instrucciones de ejecución.
panel_sql/panel.js	Carga el script, separa los seis bloques y habilita copiado y revisión.
panel_sql/styles.css	Da formato legible a los bloques y controles del panel.
pedidos_online.csv	Aporta los pedidos web que el ETL limpia, valida y reconcilia.
docker-compose.yml	Define Nginx para servir el panel localmente.